МИНИСТЕРСТВО ОБРАЗОВАНИЯ РЕСПУБЛИКИ БЕЛАРУСЬ

УЧРЕЖДЕНИЕ ОБРАЗОВАНИЯ
<БРЕСТСКИЙ ГОСУДАРСТВЕННЫЙ ТЕХНИЧЕСКИЙ УНИВЕРСИТЕТ>

ФАКУЛЬТЕТ ЭЛЕКТРОННО-ИНФОРМАЦИОННЫХ СИСТЕМ

Кафедра интеллектуальных информационных технологий

Отчет по лабораторной работе №4

Специальность АС-67

Выполнил
И. А. Якимович,
студент группы АС-67

Проверил
А. А. Крощенко,
ст. преп. кафедры ИИТ,
«___> _____2026 г.

Брест 2026

Цель: построить, обучить и оценить многослойный перцептрон (MLP) для решения задачи классификации

Вариант 1

- Iris
- **Задача:** Определить вид ириса (3 класса).
- **Архитектура:**
  - входной слой;
  - один скрытый слой с 10 нейронами (ReLU);
  - выходной слой с 3 нейронами (Softmax).
- **Эксперимент:** попробуйте добавить второй скрытый слой с 5 нейронами и сравните точность.

Выполнение:

Код программы

```
import numpy as np
import pandas as pd
import torch
import torch.nn as nn
import torch.optim as optim

from sklearn.datasets import load_iris
from sklearn.model_selection import train_test_split
from sklearn.preprocessing import StandardScaler
from sklearn.metrics import accuracy_score

torch.manual_seed(42)
np.random.seed(42)

print("   Запуск задачи классификации (Iris)")

iris = load_iris(as_frame=True)

X = iris.data.values
y = iris.target.values

print("Количество записей:", len(X))
print("Количество признаков:", X.shape[1])
print("Количество классов:", len(np.unique(y)))

print("\nРаспределение классов:")
print(pd.Series(y).value_counts().sort_index())

X_train, X_test, y_train, y_test = train_test_split(
    X,
    y,
    test_size=0.2,
    random_state=42,
    stratify=y
)

print("\nОбучающая выборка:", len(X_train))
print("Тестовая выборка:", len(X_test))

scaler = StandardScaler()

X_train = scaler.fit_transform(X_train)
X_test = scaler.transform(X_test)

X_train = torch.tensor(X_train, dtype=torch.float32)
```

```
X_test = torch.tensor(X_test, dtype=torch.float32)

y_train = torch.tensor(y_train, dtype=torch.long)
y_test = torch.tensor(y_test, dtype=torch.long)

class MLP(nn.Module):
    def __init__(self, input_size, second_layer=False):
        super().__init__()

        if second_layer:
            self.layers = nn.Sequential(
                nn.Linear(input_size, 10),
                nn.ReLU(),
                nn.Linear(10, 5),
                nn.ReLU(),
                nn.Linear(5, 3)
            )
        else:
            self.layers = nn.Sequential(
                nn.Linear(input_size, 10),
                nn.ReLU(),
                nn.Linear(10, 3)
            )

    def forward(self, x):
        return self.layers(x)

def train_model(second_layer, name):
    torch.manual_seed(42)

    model = MLP(X_train.shape[1], second_layer)

    criterion = nn.CrossEntropyLoss()
    optimizer = optim.Adam(model.parameters(), lr=0.01)

    for epoch in range(1, 201):
        model.train()

        output = model(X_train)
        loss = criterion(output, y_train)

        optimizer.zero_grad()
        loss.backward()
        optimizer.step()

        if epoch % 50 == 0:
            print(f"{name} | Эпоха {epoch}, Loss: {loss.item():.4f}")

    model.eval()

    with torch.no_grad():
        output = model(X_test)
        probabilities = torch.softmax(output, dim=1)
        predictions = torch.argmax(probabilities, dim=1).numpy()

    actual = y_test.numpy()

    accuracy = accuracy_score(actual, predictions)

    print(f"\n{name}")
    print(f"Accuracy: {accuracy:.4f}")

    return accuracy

print("\nОБУЧЕНИЕ МОДЕЛИ С ОДНИМ СКРЫТЫМ СЛОЕМ")
accuracy_1 = train_model(
    False,
    "MLP с одним скрытым слоем"
```

```
)

print("\nОБУЧЕНИЕ МОДЕЛИ С ДВУМЯ СКРЫТЫМИ СЛОЯМИ")
accuracy_2 = train_model(
    True,
    "MLP с двумя скрытыми слоями"
)

print("\nСРАВНЕНИЕ РЕЗУЛЬТАТОВ")
print(f"Один скрытый слой: {accuracy_1:.4f}")
print(f"Два скрытых слоя:  {accuracy_2:.4f}")

change = accuracy_2 - accuracy_1

print(f"Изменение Accuracy: {change:+.4f}")

if accuracy_2 > accuracy_1:
    print("Добавление второго скрытого слоя улучшило точность.")
elif accuracy_2 < accuracy_1:
    print("Добавление второго скрытого слоя снизило точность.")
else:
    print("Точность моделей не изменилась.")
```

Результат:

Запуск задачи классификации (Iris)
Количество записей: 150
Количество признаков: 4
Количество классов: 3

Распределение классов:
0 50
1 50
2 50
Name: count, dtype: int64

Обучающая выборка: 120
Тестовая выборка: 30

ОБУЧЕНИЕ МОДЕЛИ С ОДНИМ СКРЫТЫМ СЛОЕМ
MLP с одним скрытым слоем | Эпоха 50, Loss: 0.2056
MLP с одним скрытым слоем | Эпоха 100, Loss: 0.0714
MLP с одним скрытым слоем | Эпоха 150, Loss: 0.0497
MLP с одним скрытым слоем | Эпоха 200, Loss: 0.0413

MLP с одним скрытым слоем
Accuracy: 0.9667

ОБУЧЕНИЕ МОДЕЛИ С ДВУМЯ СКРЫТЫМИ СЛОЯМИ
MLP с двумя скрытыми слоями | Эпоха 50, Loss: 0.5021
MLP с двумя скрытыми слоями | Эпоха 100, Loss: 0.1436
MLP с двумя скрытыми слоями | Эпоха 150, Loss: 0.0851
MLP с двумя скрытыми слоями | Эпоха 200, Loss: 0.0639

MLP с двумя скрытыми слоями
Accuracy: 0.9667

СРАВНЕНИЕ РЕЗУЛЬТАТОВ
Один скрытый слой: 0.9667
Два скрытых слоя: 0.9667

Изменение Accuracy: +0.0000
Точность моделей не изменилась.

Вывод: В ходе лабораторной работы была разработана и обучена нейронная сеть многослойный перцептрон (MLP) для классификации ирисов на три вида: setosa, versicolor и virginica. Были реализованы и сравнены две архитектуры: с одним скрытым слоем из 10 нейронов и с двумя скрытыми слоями из 10 и 5 нейронов соответственно. В результате эксперимента обе модели показали одинаковую точность классификации, составившую 96,67%. Таким образом, добавление второго скрытого слоя не привело к улучшению точности на тестовой выборке. Цель лабораторной работы достигнута.